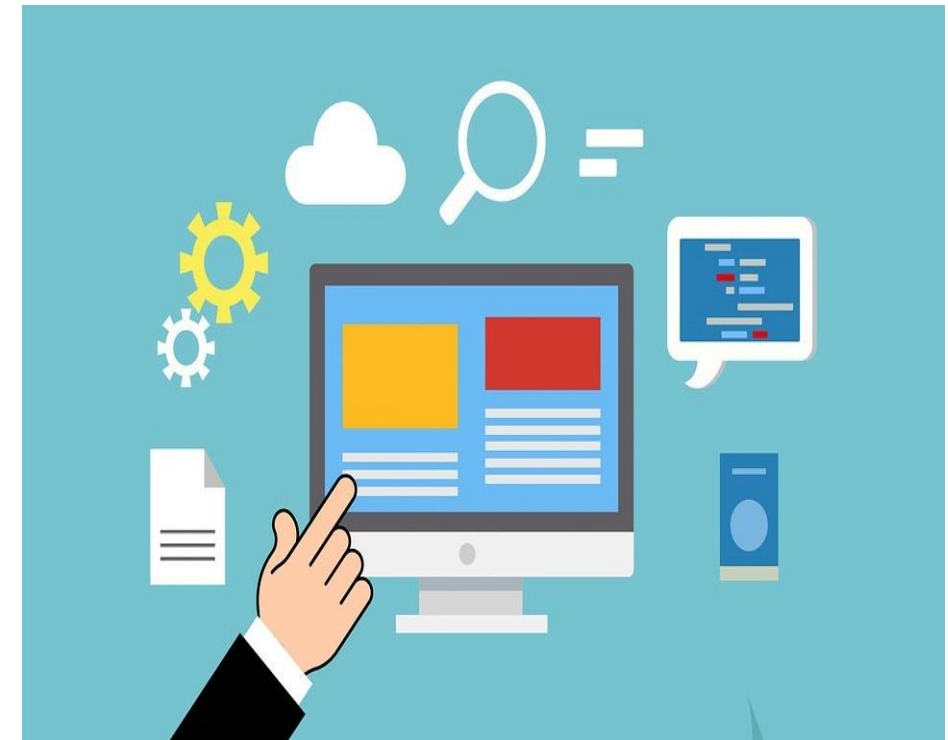


DESARROLLO DE SISTEMAS WEB

Licenciatura en Ingeniería de Ciberseguridad e
Infraestructura de Cómputo

```
else if (i==2)
{
    var atpos=inputs[i].indexof('@');
    var dotpos=inputs[i].lastindexOf('.');
    if (atpos<1 || dotpos<atpos+1 || dotpos>inputs[i].length-1)
    {
        document.getElementById('error').innerHTML+=
        'else
        document.getElementById(dv)';
    }
}
```



Fuentes de información

Unidad 3. Estándares para el desarrollo web

- Shklar Leon y Rosen Richard. Web Application, Principles, Protocols and Practices. Wiley. 2009.
- Gasston, Peter. The Modern Web: Multi-Device Web Development with HTML5, CSS3, and JavaScript. No Starch Press. 2013.
- Jennifer Niederst Robbins. Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics. O'REILLY. 2012.

Saberes teóricos

Unidad 3. Estándares para el desarrollo web

- **Unidad 3. Estándares para el desarrollo web**
- Accesibilidad de contenidos y posicionamiento en la web
- Normas de accesibilidad de aplicaciones web
- HTML semántico y posicionamiento en buscadores
- **Evolución y sintaxis de HTML**
- **Hojas de estilo en cascada**
- Lenguajes de scripting
- El modelo de DOM
- Manipulación de documentos
- Javascript Object Notation
- Javascript Asíncrono y XML (Ajax)

Evolución y sintaxis de HTML

Unidad 3. Estándares para el desarrollo web

HTML 5

III. Estándares para el desarrollo web



Example

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

Fundamentos de la Web

III. Estándares para el desarrollo web

- **HTML.** (HyperText Markup Language). Describe y define el contenido de una página Web. Generalmente trabaja en conjunto con otras tecnologías para modificar la apariencia (CSS) o la funcionalidad (Javascript).
- **CSS.** Cascading Style Sheets (CSS). Empleado para modificar la apariencia o presentación del contenido de una página Web.
- **JavaScript.** JavaScript es un robusto lenguaje de programación que puede ser aplicado a un documento HTML y usado para crear interactividad dinámica en los sitios web. Su nombre oficial es ECMAScript.



HTML

Unidad 3. Estándares para el desarrollo web

- **HTML** ("Hypertext Markup Language") no es un lenguaje de programación.
- Es un lenguaje de marcado que le dice a los navegadores web cómo estructurar las páginas web que estás visitando.
- Puede ser tan complejo o tan simple como desee el desarrollador web.
- El HTML consiste en una serie de elementos, que puedes utilizar para encerrar, delimitar o marcar diferentes partes del contenido para hacer que aparezcan de una cierta manera, o actúen de determinada forma.



Evolución de HTML

Unidad 3. Estándares para el desarrollo web

Año	Versión
1989	Tim Berners-Lee invented www
1991	Tim Berners-Lee invented HTML
1993	Dave Raggett drafted HTML+
1995	HTML Working Group defined HTML 2.0
1997	W3C Recommendation: HTML 3.2
1999	W3C Recommendation: HTML 4.01
2000	W3C Recommendation: XHTML 1.0
2008	WHATWG HTML5 First Public Draft
2012	WHATWG HTML5 Living Standard
2014	W3C Recommendation: HTML5
2016	W3C Candidate Recommendation: HTML 5.1
2017	W3C Recommendation: HTML5.1 2nd Edition
2017	W3C Recommendation: HTML5.2

World Wide Web Consortium (W3C)

Organización internacional fundada en 1994 por Tim Berners-Lee para desarrollar estándares abiertos para la Web.

WHATWG

(Web Hypertext Application Technology Working Group)

Grupo formado en 2004 por ingenieros de:

- Mozilla
- Apple
- Opera (posteriormente también Google)

En 2019:

W3C y WHATWG firmaron un acuerdo: WHATWG sería el único responsable de mantener el estándar HTML.

Desde entonces:

- HTML es oficialmente un Living Standard.
- W3C colabora pero no mantiene versión paralela.

Partes de un documento HTML

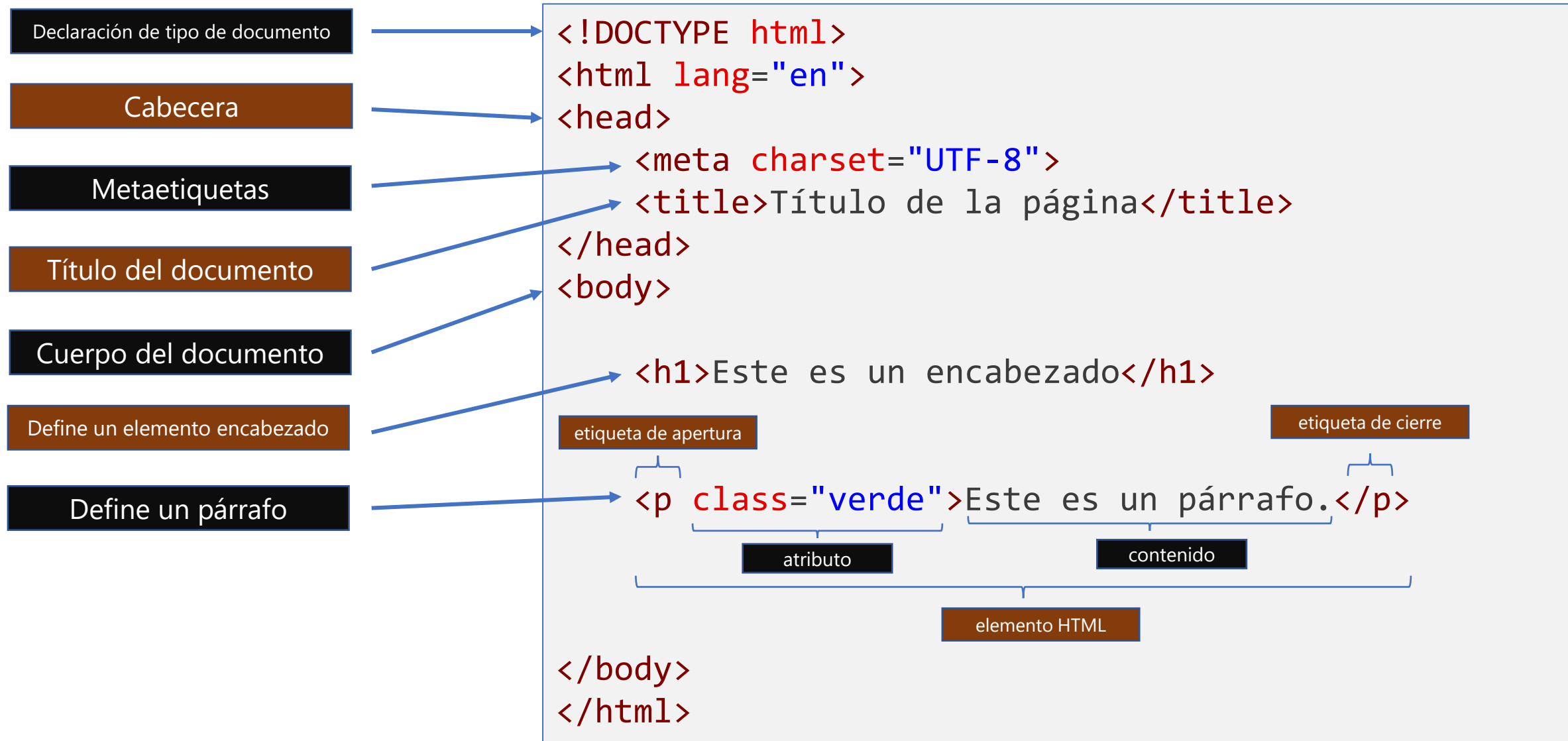
Unidad 3. Estándares para el desarrollo web

- Además de todo esto, **un documento HTML ha de estar delimitado por la etiqueta HTML**. Dentro de este documento, podemos asimismo distinguir dos partes principales:
- **La cabecera, delimitada por la etiqueta HEAD**, donde colocaremos etiquetas de índole informativo, como por ejemplo el título de nuestra página. El contenido de la cabecera no suele aparecer en el cuerpo de la página, pero sirve a los navegadores y otros sistemas para encontrar información útil para entender y procesar el documento.
- **El cuerpo, delimitado por la etiqueta BODY**, que será donde colocaremos nuestro texto e imágenes definidas a su vez por otras etiquetas.

```
<!DOCTYPE html>
<html>
<head>
<title>Título de la página</title>
</head>
<body>
<h1>Este es el encabezado</h1>
<p>Este es un párrafo</p>
</body>
</html>
```

Partes de un documento HTML

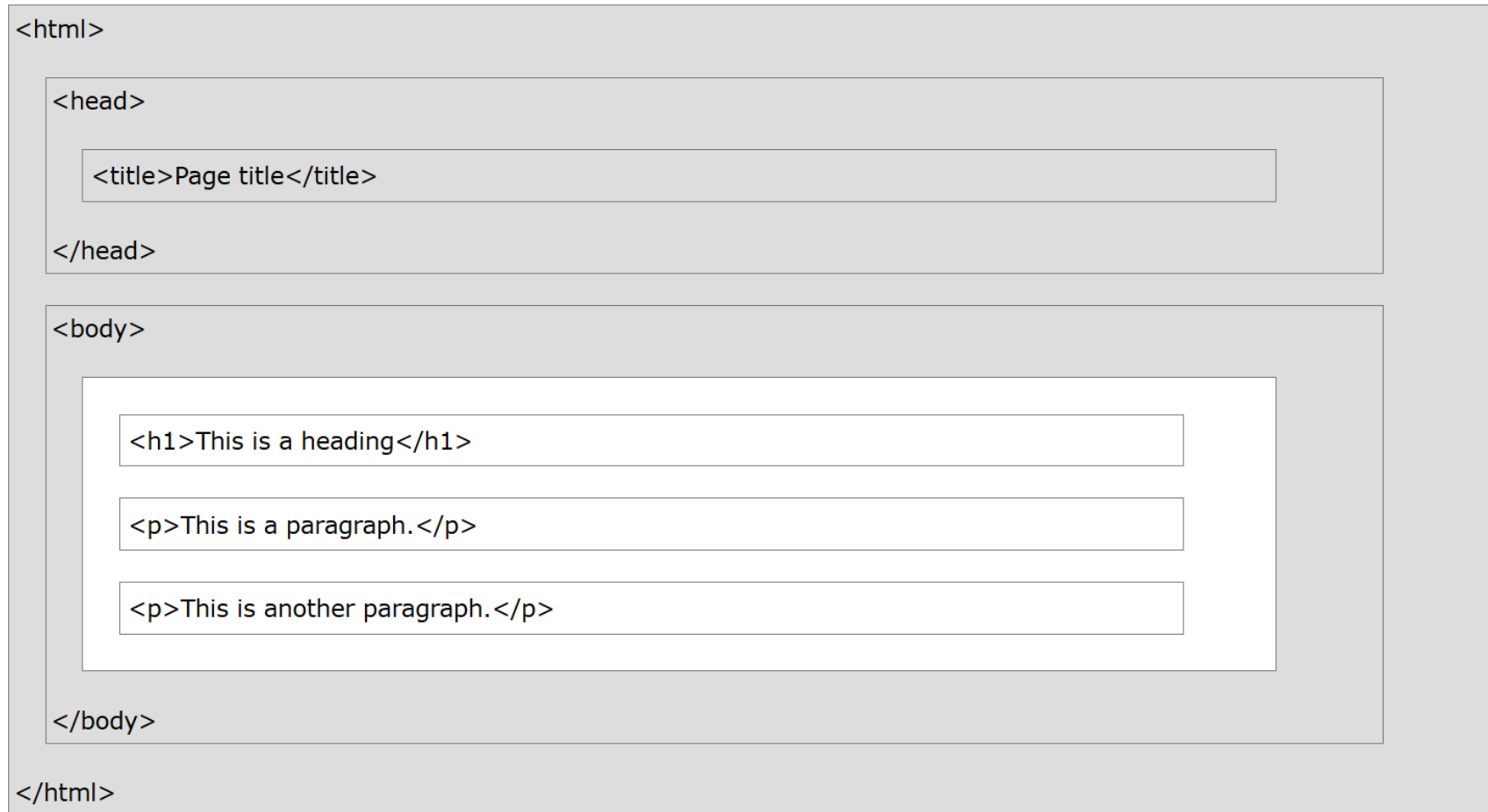
Unidad 3. Estándares para el desarrollo web



Partes de un documento HTML

Unidad 3. Estándares para el desarrollo web

- Los elementos HTML son los constructores de la página.



Anatomía de una etiqueta HTML

Unidad 3. Estándares para el desarrollo web

- La etiqueta presenta frecuentemente dos partes, su apertura y cierre, y se encierran ambas partes entre símbolos "menor que" y "mayor que" como se muestra a continuación:

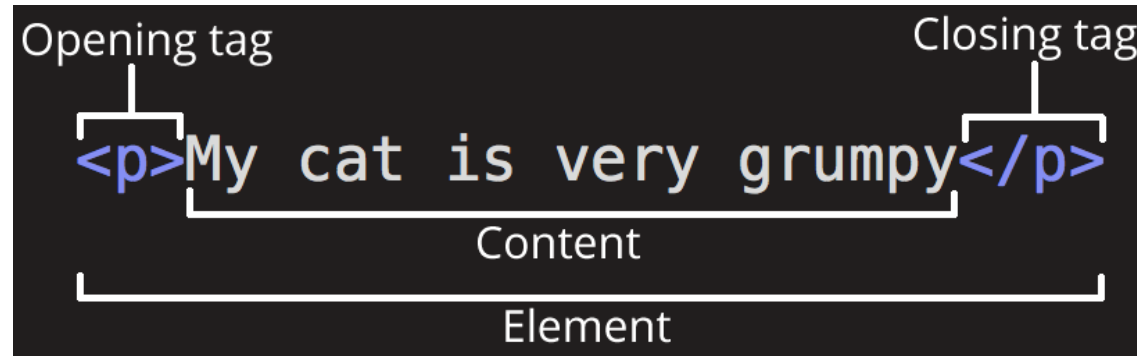
<etiqueta> </etiqueta>

- Las etiquetas en HTML NO distinguen entre mayúsculas y minúsculas, es decir, pueden escribirse en mayúsculas o minúsculas.

<https://www.w3schools.com/tags/default.asp>

Anatomía de una etiqueta HTML

Unidad 3. Estándares para el desarrollo web



- **La etiqueta de apertura:** Consiste en el nombre del elemento (en este caso, p), envuelto en paréntesis angulares de apertura y cierre . Esto indica dónde comienza a surtir efecto el elemento, en este caso dónde está el comienzo del párrafo.
- **La etiqueta de cierre:** es la misma que la etiqueta de apertura, excepto que incluye una barra diagonal antes del nombre del elemento. Esto indica dónde termina el elemento, en este caso dónde está el final del párrafo. No incluir una etiqueta de cierre es un error común de principiante y puede dar lugar a resultados extraños.
- **El contenido:** este es el contenido del elemento, que en este caso es solo texto.
- **El elemento:** la etiqueta de apertura, más la etiqueta de cierre, más el contenido equivale al elemento.

Elementos de bloque y elementos en línea

Unidad 3. Estándares para el desarrollo web

- Los **elementos de bloque** forman un bloque visible en la página.
 - Un elemento a nivel de bloque siempre comienza en una nueva línea y los navegadores agregan automáticamente algún espacio (un margen) antes y después del elemento.
 - Un elemento a nivel de bloque siempre ocupa todo el ancho disponible (se extiende hacia la izquierda y la derecha tanto como puede).
 - Dos elementos de bloque comúnmente utilizados son: `<p>` y `<div>`.

```
<p style="border: 1px solid black">Hello World</p>  
<div style="border: 1px solid black">Hello World</div>
```

Hello World

Hello World

- Otros elementos de bloque son: [`<div>`](#), [`<hr>`](#), [`<table>`](#)

Elementos de bloque y elementos en línea

Unidad 3. Estándares para el desarrollo web

- Los **elementos en línea** están contenidos dentro de elementos de bloque y delimitan solo pequeñas partes del contenido del documento.
 - Un elemento en línea no comienza en una nueva línea.
 - Un elemento en línea solo ocupa el ancho necesario.

```
<p>This is an inline span <span style="border: 1px solid black">Hello  
World</span> element inside a paragraph.</p>
```

This is an inline span Hello World element inside a paragraph.

- Otros elementos en línea son:
 - [<a>](#)
 - [](#)
 - [](#)

Un elemento en línea no puede contener un elemento en bloque

Elementos vacíos

Unidad 3. Estándares para el desarrollo web

- No todos los elementos siguen el patrón de etiqueta de apertura, contenido y etiqueta de cierre. Algunos elementos consisten solo en una etiqueta única, que se utiliza generalmente para insertar/incrustar algo en el documento en el lugar donde se le quiere incluir.

```

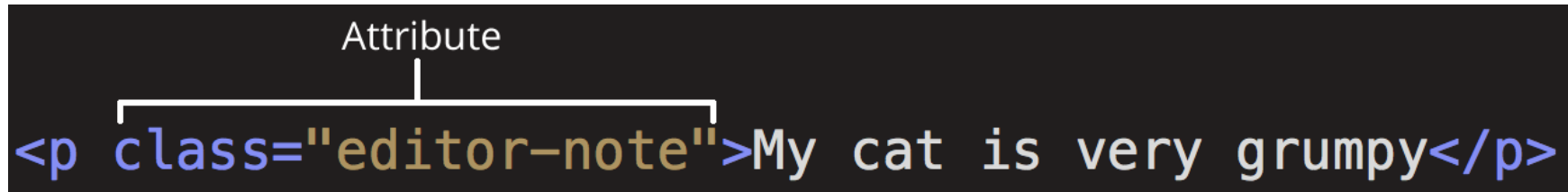
```

```
<input type="text" id="lname" name="lname">
```

```
<p>To force<br> line breaks<br> in a text,<br> use the br<br> element.</p>
```


Atributos

Unidad 3. Estándares para el desarrollo web



```
<p class="editor-note">My cat is very grumpy</p>
```

- Los atributos contienen información extra sobre el elemento y no se muestra en el contenido. En este caso, el atributo class asigna al elemento un identificador que se puede utilizar para dotarlo de información de estilo.
- Un atributo debería tener:
 - Un espacio entre este y el nombre del elemento. (Para un elemento con más de un atributo, los atributos también deben estar separados por espacios).
 - El nombre del atributo, seguido por un signo igual.
 - Un valor del atributo, rodeado de comillas de apertura y cierre.

Atributos

Unidad 5. Estándares para el desarrollo web

- Aunque cada una de las etiquetas HTML define sus propios atributos, algunos de los atributos son comunes a muchas o casi todas las etiquetas.
- Los atributos comunes se dividen en cuatro grupos según su funcionalidad:
- **Atributos básicos:** se pueden utilizar prácticamente en todas las etiquetas HTML.
- **Atributos para internacionalización:** los utilizan las páginas que muestran sus contenidos en varios idiomas.
- **Atributos de eventos:** sólo se utilizan en las páginas web dinámicas creadas con JavaScript.
- **Atributos de foco:** relacionados principalmente con la accesibilidad de los sitios web.

```
<script src="myscripts.js"></script>
```

```

```

```
<button name="subject" type="submit" value="HTML">HTML</button>
```

Atributo	Descripción
lang = "codigo de idioma"	Indica el idioma del elemento mediante un código predefinido
xml:lang = "codigo de idioma"	Indica el idioma del elemento mediante un código predefinido
dir	Indica la dirección del texto (útil para los idiomas que escriben de derecha a izquierda)

Atributos para internacionalización

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
```

```
<!DOCTYPE html>
<html>
<body>

<p dir="rtl">Escribe este texto de derecha a izquierda!</p>
```

<https://www.w3.org/International/questions/qa-html-language-declarations.es>

Atributo	Descripción
id = "texto"	Establece un identificador único a cada elemento dentro de una página HTML
class = "texto"	Establece la clase CSS que se aplica a los estilos del elemento
style = "texto"	Establece de forma directa los estilos CSS de un elemento
title = "texto"	Establece el título a un elemento (mejora la accesibilidad y los navegadores lo muestran cuando el usuario pasa el ratón por encima del elemento)

Atributos básicos

Atributo	Descripción	Elementos que pueden usarlo
onblur	Deseleccionar el elemento	<button>, <input>, <label>, <select>, <textarea>, <body>
onchange	Deseleccionar un elemento que se ha modificado	<input>, <select>, <textarea>
onclick	Pinchar y soltar el ratón	Todos los elementos
ondblclick	Pinchar dos veces seguidas con el ratón	Todos los elementos
onfocus	Seleccionar un elemento	<button>, <input>, <label>, <select>, <textarea>, <body>
onkeydown	Pulsar una tecla (sin soltar)	Elementos de formulario y <body>

Atributos de eventos

Atributo	Descripción	Elementos que pueden usarlo
onkeypress	Pulsar una tecla	Elementos de formulario y <body>
onkeyup	Soltar una tecla pulsada	Elementos de formulario y <body>
onload	La página se ha cargado completamente	<body>
onmousedown	Pulsar (sin soltar) un botón del ratón	Todos los elementos
onmousemove	Mover el ratón	Todos los elementos
onmouseout	El ratón " <i>sale</i> " del elemento (pasa por encima de otro elemento)	Todos los elementos
onmouseover	El ratón " <i>entra</i> " en el elemento (pasa por encima del elemento)	Todos los elementos
onmouseup	Soltar el botón que estaba pulsado en el ratón	Todos los elementos

Atributos de eventos

Atributo	Descripción	Elementos que pueden usarlo
onmouseup	Soltar el botón que estaba pulsado en el ratón	Todos los elementos
onreset	Inicializar el formulario (borrar todos sus datos)	<form>
onresize	Se ha modificado el tamaño de la ventana del navegador	<body>
onselect	Seleccionar un texto	<input>, <textarea>
onsubmit	Enviar el formulario	<form>
onunload	Se abandona la página (por ejemplo al cerrar el navegador)	<body>

Atributos de eventos

Comentarios

Unidad 3. Estándares para el desarrollo web

- Los comentarios son ignorados por el navegador y, por tanto, son invisibles para el usuario. El propósito de los comentarios es permitirte incluir notas en el código para explicar tu lógica o codificación.

Para convertir en un comentario una sección de contenido de tu archivo HTML, debes delimitarlo con los marcadores especiales `<!--` y `-->`. Por ejemplo:

```
HTML Play   
  
<p>No soy un comentario</p>  
  
<!-- <p>¡Yo sí!</p> -->
```

Como puedes ver a continuación, el primer párrafo aparece, pero el segundo no.

```
Play   
  
No soy un comentario
```


HTML semántico

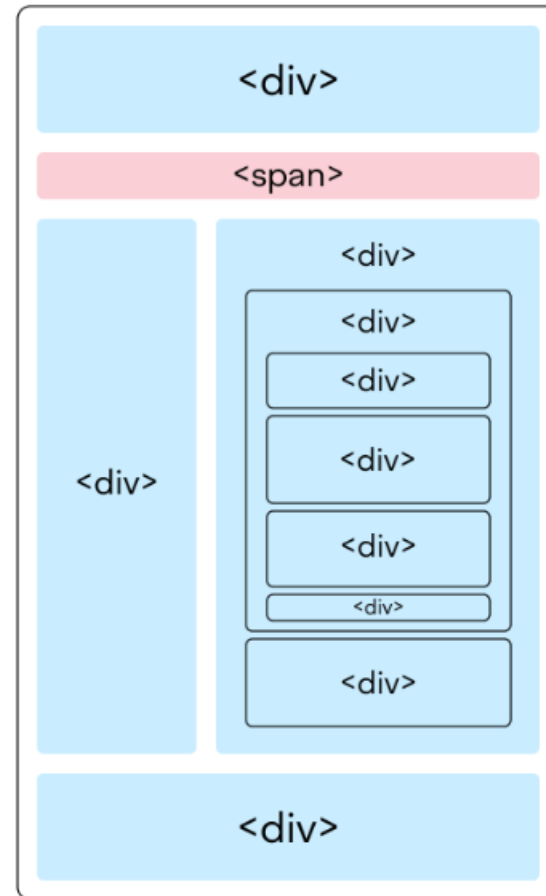
Unidad 3. Estándares para el desarrollo web

- El HTML semántico, también conocido como marcado semántico, engloba las etiquetas HTML que transmiten el significado (o semántica) de su contenido.
- Al añadir etiquetas HTML semánticas a tus páginas, proporcionas información adicional a los buscadores sobre la funcionalidad e importancia de las secciones de tus páginas.
- Los elementos semánticos HTML son aquellos que describen claramente su significado de una manera legible por humanos y máquinas. Elementos tales como `<header>` , `<footer>` y `<article>` son considerados semánticos porque describen con precisión el propósito del elemento y el tipo de contenido que hay dentro de ellos

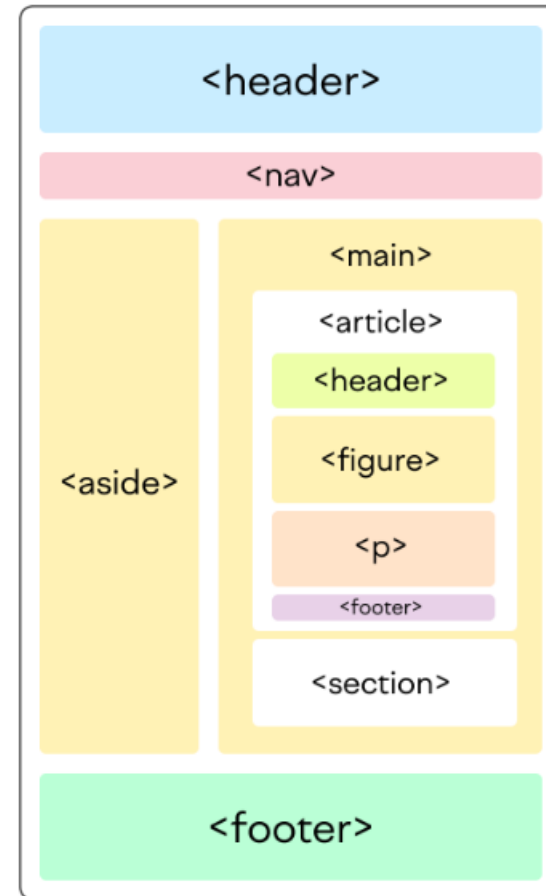
HTML semántico

Unidad 3. Estándares para el desarrollo web

HTML no semántico



HTML semántico



Accesibilidad: ARIA y HTML semántico

Unidad 3. Estándares para el desarrollo web

- Accessible Rich Internet Applications (ARIA) es un conjunto de roles y atributos que definen formas de hacer que el contenido web y las aplicaciones web (especialmente aquellas desarrolladas con JavaScript) sean más accesibles para las personas con discapacidades.
- ARIA complementa HTML para que las interacciones y los widgets comúnmente utilizados en las aplicaciones puedan transferirse a tecnologías de asistencia cuando no existe un mecanismo adecuado. Por ejemplo, ARIA permite widgets JavaScript accesibles, sugerencias de formularios y mensajes de error, actualizaciones de contenido en tiempo real y mucho más.

Accesibilidad: ARIA y HTML semántico

Unidad 3. Estándares para el desarrollo web

- ARIA (Accessible Rich Internet Applications) fue creada y desarrollada por el **Consorcio World Wide Web (W3C)**, específicamente a través de su iniciativa de accesibilidad, la **Web Accessibility Initiative (WAI)**.
- WAI-ARIA 1.0 se convirtió en una Recomendación completa del W3C el 20 de marzo de 2014, aunque los borradores existían desde 2008.
- Es un conjunto de roles y atributos que definen formas de hacer que el contenido web y las aplicaciones web (especialmente aquellas desarrolladas con JavaScript) sean más accesibles para las personas con discapacidades.
- ARIA complementa HTML para que las interacciones y los widgets comúnmente utilizados en las aplicaciones puedan transferirse a tecnologías de asistencia cuando no existe un mecanismo adecuado. Por ejemplo, ARIA permite widgets JavaScript accesibles, sugerencias de formularios y mensajes de error, actualizaciones de contenido en tiempo real y mucho más.

Accesibilidad: ARIA y HTML semántico

Unidad 3. Estándares para el desarrollo web

- ARIA no reemplaza HTML semántico y es recomendable usarlo en los siguientes casos:

1. Se crean componentes dinámicos personalizados

- Menú desplegable avanzado
- Acordeón
- Pestañas
- Modal

2. Se necesita comunicar estado dinámico

```
<button aria-expanded="true">
```

3. Se necesita describir relación entre elementos

```
<section aria-labelledby="titulo">
```

4. Se trabaja con aplicaciones SPA (Single Page Application)

- React, Angular, Vue suelen requerir ARIA en ciertos componentes personalizados.

"No tener ARIA es mejor que tener una mala ARIA"

Accesibilidad: ARIA y HTML semántico

Unidad 3. Estándares para el desarrollo web

3. Se necesita describir relación entre elementos

```
<section aria-labelledby="titulo">
```

- aria-labelledby es un **atributo ARIA (Accessible Rich Internet Applications)** que se utiliza para **asociar un elemento con otro elemento que actúa como su etiqueta o descripción principal**.
- Permite que tecnologías de asistencia (como lectores de pantalla) identifiquen correctamente el nombre accesible de un elemento.

Accesibilidad: ARIA y HTML semántico

Unidad 3. Estándares para el desarrollo web

Cómo funciona

- El atributo recibe como valor el **id de otro elemento**.

```
<section aria-labelledby="titulo-servicios">  
  <h2 id="titulo-servicios">Servicios</h2>  
  <p>Lista de servicios institucionales...</p>  
</section>
```

- El <section> no tiene texto propio que lo identifique.
- aria-labelledby="titulo-servicios" le indica al lector de pantalla:
"El nombre de esta sección es el contenido del elemento con id titulo-servicios".
- Así, el lector de pantalla anunciará:
"Sección Servicios"

Accesibilidad: ARIA y HTML semántico

Unidad 3. Estándares para el desarrollo web

aria-label

aria-label es un atributo de **ARIA (Accessible Rich Internet Applications)** que permite definir **un nombre accesible explícito para un elemento**, cuando ese nombre no está presente visualmente en la interfaz.

Sirve para:

- Proporcionar una etiqueta accesible a lectores de pantalla.
- Describir botones con solo íconos.
- Nombrar regiones sin encabezado visible.
- Mejorar accesibilidad sin modificar el diseño visual.

Accesibilidad: ARIA y HTML semántico

Unidad 3. Estándares para el desarrollo web

Sintaxis

`aria-label="Texto descriptivo"`

El "Texto descriptivo":

- No se muestra en pantalla.
- Es leído por lectores de pantalla.
- Se convierte en el **nombre accesible del elemento**.

```
<button aria-label="Buscar">
```



```
</button>
```

El lector anunciará:

"Buscar, botón"

Accesibilidad: ARIA y HTML semántico

Unidad 3. Estándares para el desarrollo web

Comparación aria-labelledby / aria-label

```
<section id="contacto" aria-labelledby="contacto-titulo">  
  <h2 id="contacto-titulo">Contacto y reporte de incidente</h2>
```

```
<nav aria-label="Navegación principal">
```

¿Por qué?

Porque no hay un <h2> visible que diga "Navegación principal".

Actividad en clase

Actividad en clase

Nombre: DSW-A05. Página HTML- Centro de Cómputo FEI.

Instrucciones:

Realiza la práctica indicada en la actividad de Eminus.

Coloca el **reporte de práctica** en el apartado de su actividad.



Hojas de estilo en cascada

Estándares para el desarrollo web

Hojas de estilo en cascada

Tecnologías web

- Del inglés *Cascading Style Sheets* u hoja de estilos en cascada (CSS)
- Es un *pseudo-lenguaje* usado para definir la presentación de un documento estructurado escrito en **HTML o XML** (y por extensión en XHTML).
- Es un mecanismo simple que describe cómo se va a mostrar un documento en la pantalla, o cómo se va a imprimir.
- CSS fue desarrollado por W3C (**World Wide Web Consortium**) en 1996 por una razón muy sencilla: HTML no fue diseñado para tener etiquetas que ayuden a formatear la página. HTML está hecho solo para escribir el marcado para el sitio.



Características

Hojas de estilo en cascada

- Se utiliza para dar estilo a los documentos.
- Un estilo define la forma de mostrar los elementos de la página Web.
- Separa el contenido de la presentación.
- Código HTML más liviano y comprensible.

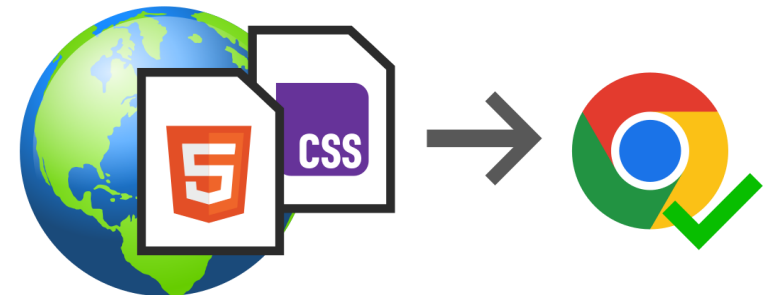


Evolución

Hojas de estilo en cascada

- En 1996, el W3C publicó la primera recomendación oficial, "CSS nivel 1".
- En 1998, el W3C publicó la segunda recomendación oficial, "CSS nivel 2".
- La versión utilizada actualmente es CSS 2.1 (2011).
 - Se publican cada año añadidos o snapshots.
- CSS nivel 3, continúa en desarrollo, adopta un enfoque modular, donde cada módulo define una parte del CSS, en lugar de definir una única especificación monolítica.

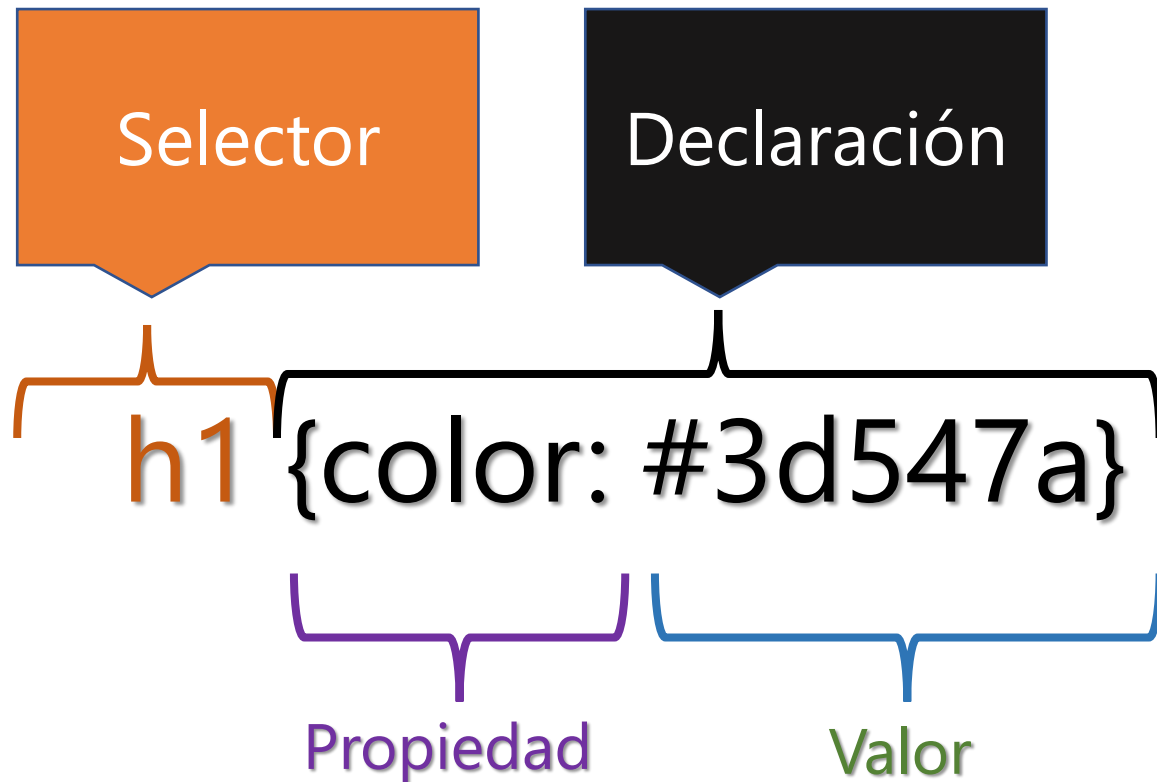
<https://www.w3.org/Style/CSS/> o <https://www.w3.org/TR/css-2026/>



¿Cómo funciona?

Hojas de estilo en cascada

- Funciona a base de reglas, establecidas para los elementos de la página Web.
- Una hoja de estilos está compuesta por una o más reglas.



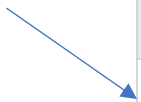
La declaración tiene dos partes: propiedad y valor.

Selectores

Hojas de estilo en cascada

- Un selector CSS es la primera parte de una regla CSS.
- Es un patrón de elementos y otros términos que indican al navegador qué elementos HTML se seleccionan para aplicarles una regla que incluye los valores de las propiedades CSS.

SELECTOR UNIVERSAL



Selector	Example	Example description
<u>#id</u>	#firstname	Selects the element with id="firstname"
<u>.class</u>	.intro	Selects all elements with class="intro"
<u>element.class</u>	p.intro	Selects only <p> elements with class="intro"
<u>*</u>	*	Selects all elements
<u>element</u>	p	Selects all <p> elements
<u>element,element,..</u>	div, p	Selects all <div> elements and all <p> elements

Tipos de selectores

Hojas de estilo en cascada

- **Selectores de tipo, de clase y de ID.** Este grupo incluye selectores que delimitan un elemento HTML, como por ejemplo un `<h1>`

https://www.w3schools.com/css/css_selectors.asp

Ejemplos:

Todos los elementos <code><p></code> de la página estarán alineados al centro, con un color de texto rojo:	Se aplicará al elemento HTML con <code>id="para1"</code>	Todos los elementos HTML con <code>clase="center"</code> serán rojos y estarán alineados al centro	Solo los elementos <code><p></code> con <code>clase="center"</code> serán rojos y estarán alineados al centro:
<pre>p { text-align: center; color: red; }</pre>	<pre>#para1 { text-align: center; color: red; }</pre>	<pre>.center { text-align: center; color: red; }</pre>	<pre>p.center { text-align: center; color: red; }</pre>

Tipos de selectores

Hojas de estilo en cascada

- **Selectores de atributo.** Este grupo de selectores te proporciona diferentes formas de seleccionar elementos según la presencia de un atributo determinado en un elemento.

https://www.w3schools.com/css/css_attribute_selectors.asp

Ejemplos:

Selecciona todos los elementos <a> con un atributo de destino:	Selecciona todos los elementos <a> con un atributo target="_blank":	Selecciona elementos con el valor "top" en el atributo class.
<pre>a[target] { background-color: yellow; }</pre>	<pre>a[target="_blank"] { background-color: yellow; }</pre>	<pre>[class="top"] { background: yellow; }</pre>

Tipos de selectores

Hojas de estilo en cascada

- **Las pseudoclases y los pseudoelementos.** Este grupo de selectores incluye pseudoclases, que aplican estilo a ciertos estados de un elemento. La pseudoclase `:hover`, por ejemplo, selecciona un elemento solo cuando se le pasa el ratón por encima.

https://www.w3schools.com/css/css_pseudo_classes.asp

Ejemplos:

Da estilo a los hipervínculos donde pasa el mouse	Da estilo a un campo de entrada cuando recibe el foco: <code>:focus</code>	Da estilo al elemento <code><a></code> con la clase "highlight", cuando pasa el mouse encima	El selector coincide con todos los elementos <code><p></code> que son el primer hijo de cualquier elemento
<pre>a:hover { color: hotpink; }</pre>	<pre>input:focus { background-color: yellow; }</pre>	<pre>a.highlight:hover { color: #ff0000; }</pre>	<pre>p:first-child { color: blue; }</pre>

Tipos de selectores

Hojas de estilo en cascada

Combinadores. El último grupo de selectores combina otros selectores con el fin de delimitar elementos de nuestros documentos.

https://www.w3schools.com/css/css_combinators.asp

Ejemplos:

Selecciona todos los elementos <p> dentro de los elementos <div>	Selecciona todos los elementos <p> que son hijos directos de <div>
<pre>div p { background-color: yellow; }</pre>	<pre>div > p { background-color: yellow; }</pre>

Implementación

Hojas de estilo en cascada

- **Hay tres formas de aplicar estilos** CSS en un documento HTML: en línea, incrustado en la cabecera y mediante hojas de estilo externas.

1. CSS en línea: Los estilos en línea son declaraciones CSS que se integran en las etiquetas HTML mediante el atributo style. Este método tan solo afecta al elemento en el que se integra el código. El CSS en línea es complicado de entender y mantener ya que mezcla los estilos CSS con el código HTML.

- **Ejemplo:**

- `<p style="color:red">Párrafo de color rojo.</p>`

No se
recomienda

Implementación

Hojas de estilo en cascada

2. Incrustado en la cabecera: Esta forma es a veces utilizada; consiste en incluir el código CSS en la cabecera del sitio (etiquetas head del documento HTML), concretamente dentro de las etiquetas `<style>` y `</style>`

Utilizando el elemento
`<style>` en el documento

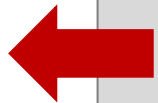
```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8">
    <title>Hoja de estilo interna</title>
    <style type="text/css">
      h1 {
        font-family: Helvetica, Geneva, Arial, sans-serif;
        color: #3d547a;
        font-size: 14px;
      }
      h2 {
        font-family: Helvetica, Geneva, Arial, sans-serif;
        color: #000000;
        font-size: 12px;
      }
    </style>
  </head>
  <body>
    <h1>Aquí se aplicará el estilo de letra para el Título h1</h1>
    <h2>Aquí se aplicará el estilo de letra para el Título h1</h2>
  </body>
</html>
```

Implementación

Hojas de estilo en cascada

3. Hoja de estilos externa: Esta es la forma más versátil y la única que recomiendo; un archivo externo contiene todo nuestro código CSS de manera que es completamente independiente el código propio de la página web de su código de formato.

Utilizando el
elemento `<link>`,
css externa



```
<!DOCTYPE html>
<html>
  <head>
    <title>Título</title>
    <link rel="stylesheet" type="text/css" href="css/estilos.css" />
  </head>
  <body>
  </body>
</html>
```


Reglas en CSS

Hojas de estilo en cascada

Características de las reglas CSS: Cascada, Especificidad, Herencia.

- **Cascada.** En un primer nivel de simplicidad, la cascada en las hojas de estilo significa que el orden de las reglas importa en CSS: cuando dos reglas tienen la misma especificidad, se aplica la que aparece en último lugar en el CSS.
- Ejemplo:

CSS

```
h1 {  
    color: red;  
}  
  
h1 {  
    color: blue;  
}
```

HTML

```
<h1>Este es mi encabezado.</h1>
```

Resultado:

Este es mi encabezado

Reglas en CSS

Hojas de estilo en cascada

- **Especificidad.** La especificidad es el modo que tiene el navegador de decidir qué regla se aplica si diversas reglas tienen selectores diferentes pero podrían aplicarse a un mismo elemento. Básicamente, la especificidad mide cuán específica es la selección de un selector:
- Un selector de elemento es menos específico por lo que presenta una puntuación más baja en especificidad.
- Un selector de clase es más específico y por tanto recibe una puntuación mayor.
- Ejemplo:

CSS

```
.cabecera {  
    color: red;  
}  
  
h1 {  
    color: blue;  
}
```

HTML

```
<h1 class="cabecera">Este es mi encabezado.</h1>
```

Resultado:

Este es mi encabezado

Especificidad: Prioridades

Hojas de estilo en cascada

Es un algoritmo que determina qué declaración de estilo se aplica finalmente a un elemento.

Si dos o más reglas CSS apuntan al mismo elemento, la declaración con mayor especificidad "ganará" y ese estilo se aplicará al elemento HTML.

Orden de ejecución descendente, pero **CUIDADO** en la prioridad de las reglas:

1. Id
2. Clase
3. Nombre de elemento



```
#miparrafo {  
  color: blue;  
  text-align: center;  
}
```

Resultado:

¡Hola mundo!

```
.subtitulo {  
  color: green;  
  text-align: center;  
}
```

```
p {  
  text-align: center;  
  color: red;  
}
```

```
<p id="miparrafo" class=".subtitulo">¡Hola mundo!</p>
```

Especificidad: Prioridades

Hojas de estilo en cascada

La regla !important se utiliza para dar la máxima prioridad al valor de una propiedad específica.

Anulará TODAS las reglas de estilo anteriores para esa propiedad específica en ese elemento!

```
selector {  
  property: value !important;  
}
```

https://www.w3schools.com/css/css_important.asp

```
p {  
  background-color: yellow !important;  
}  
  
#myid {  
  background-color: blue;  
}  
  
.myclass {  
  background-color: gray;  
}
```

Reglas en CSS

Hojas de estilo en cascada

- **Herencia.** Algunos valores de las propiedades CSS que se han establecido para los elementos padre los heredan los elementos hijo, pero otros no.
- Ejemplo:

CSS

```
body {  
    color: blue;  
}
```

```
span {  
    color: black;  
}
```

HTML

```
<h1>Este es mi encabezado</h1>  
<p><span>Este es un párrafo.</span></p>
```

Resultado:

Este es mi encabezado

Este es un párrafo

Unidades de medida CSS

Hojas de estilo en cascada

- Las medidas en CSS se emplean, entre otras, para definir la altura, anchura y márgenes de los elementos y para establecer el tamaño de letra del texto.
- Todas las medidas se indican como un valor numérico entero o decimal seguido de una unidad de medida (sin ningún espacio en blanco entre el número y la unidad de medida).

```
h1 {  
  font-size: 40px;  
}  
  
p {  
  font-size: 20px;  
  line-height: 50px;  
}
```

Existen dos tipos de unidades:

- Absolutas.
- Relativas.

Unidades de medida CSS

Hojas de estilo en cascada

• Absolutas

Una longitud expresada como absoluta aparecerá exactamente con ese tamaño, por lo que no se recomienda el uso de unidades de longitud absoluta en pantalla.

Son más útiles en una salida de formato impreso.

Unidad	Nombre	Equivale a
cm	Centímetros	1cm = 96px/2,54
mm	Milímetros	1mm = 1/10 de 1cm
Q	Cuartos de milímetros	1Q = 1/40 de 1cm
in	Pulgadas	1in = 2,54cm = 96px
pc	Picas	1pc = 1/6 de 1in
pt	Puntos	1pt = 1/72 de 1in
px	Píxeles	1px = 1/96 de 1in

Unidades de medida CSS

Hojas de estilo en cascada

- **Relativas**

Las unidades de longitud relativa especifican una longitud relativa a otra propiedad de longitud, por ejemplo relativo al font-size de un elemento padre, al ancho/altura de un contenedor padre o al ancho/altura del viewport.

Se recomienda utilizarlas sobre todo para Responsive Web Design (RWD).

*viewport (Ventana gráfica) = Tamaño de la ventana del navegador. Si la ventana gráfica tiene 50 cm de ancho, $1\text{vw} = 0,5\text{ cm}$.

Unit	Description
em	Relativo al tamaño de fuente del elemento (2em significa 2 veces el tamaño de la fuente actual)
ex	Relativo a la altura x de la fuente actual (rara vez se utiliza)
ch	Relativo al ancho del "0" (cero)
rem	Relativo al tamaño de fuente del elemento raíz
vw	Relativo al 1% del ancho del viewport* (ventana gráfica)
vh	Relativo al 1% de la altura de viewport*
vmin	Relativo al 1% de la dimensión más pequeña de viewport*
vmax	Relativo al 1% de la dimensión más grande de viewport*
%	Relativo al elemento padre

Unidades de medida CSS

Hojas de estilo en cascada

Relativas: ems y rems. Son las empleadas con mayor frecuencia.

- La unidad **em** significa «el tamaño de letra de mi elemento padre».
- **1em** equivale al tamaño de fuente actual del elemento padre.

Si el padre tiene `font-size: 16px`, entonces:

`1em = 16px`

`2em = 32px`

`0.5em = 8px`

En el siguiente ejemplo:

CSS:

```
.contenedor {  
  font-size: 20px;  
}
```

```
.contenedor p {  
  font-size: 1.5em;  
}
```

HTML:

```
<div class="contenedor">  
  <p>Este es un párrafo con  
  tamaño relativo.</p>  
</div>
```

1. El elemento padre (`.contenedor`) tiene `font-size: 20px`.
2. El párrafo tiene `font-size: 1.5em`.
3. El valor se calcula así:
 $1.5 \times 20px = 30px$

Por lo tanto, el párrafo se mostrará con:
30px

Unidades de medida CSS

Hojas de estilo en cascada

La unidad **rem** significa «el tamaño de letra del elemento raíz». ('rem' viene de «root em»).

- **1rem** equivale al tamaño de fuente del elemento raíz (html).
- A diferencia de em, **no depende del elemento padre inmediato**, sino siempre del tamaño definido en html.
- Por defecto, la mayoría de los navegadores utilizan:

```
html {  
  font-size: 16px;  
}
```

- Por lo tanto:

Valor	Equivalencia
1rem	16px
1.5rem	24px
2rem	32px

Unidades de medida CSS

Hojas de estilo en cascada

Se tiene el siguiente ejemplo:

CSS:

```
html {  
  font-size: 16px;  
}
```

```
.contenedor p {  
  font-size: 1.5rem;  
}
```

HTML:

```
<div class="contenedor">  
  <p>Este es un párrafo con tamaño  
relativo a la raíz.</p>  
</div>
```

1. El tamaño base del documento es:

16px

2. El párrafo tiene:

1.5rem

Entonces:

$1.5 \times 16\text{px} = 24\text{px}$

El texto del párrafo se mostrará con **24px**.

Unidades de medida CSS

Hojas de estilo en cascada

- **Relativas: porcentajes.** La unidad de medida porcentual es la que se usa por defecto en los elementos HTML en donde de manera predeterminada cada elemento de bloque usa un ancho del 100%, es por eso que cuando achicamos la ventana del navegador con una página que no tenga estilos, la página se adapta, ya que siempre usará el ancho total visible.

En muchos casos, un porcentaje es tratado de la misma manera que una longitud. Lo que sucede con los porcentajes es que siempre se establecen en relación con otro valor. Por ejemplo, si estableces el atributo `font-size` de un elemento como un porcentaje, será un porcentaje del `font-size` del elemento padre. Si usas un porcentaje para un valor `width`, será un porcentaje del atributo `width` del elemento padre.

Unidades de medida CSS

Hojas de estilo en cascada

Relativas: porcentajes. Ejemplo 1 – Porcentaje en tamaño de fuente

CSS:

```
.contenedor {  
  font-size: 20px;  
}
```

```
.contenedor p {  
  font-size: 150%;  
}
```

HTML:

```
<div class="contenedor">  
  Texto del contenedor  
  <p>Este párrafo tiene un  
  tamaño relativo.</p>  
</div>
```

1. El contenedor tiene:
20px
2. El párrafo usa:
150%
3. Entonces:
 $150\% \text{ de } 20\text{px} = 30\text{px}$
El tamaño final del párrafo será:
30px

Unidades de medida CSS

Hojas de estilo en cascada

Relativas: porcentajes. Ejemplo 2 – Porcentaje en ancho de un elemento

CSS:

```
.contenedor {  
  width: 600px;  
  border: 1px solid black;  
}
```

```
.caja {  
  width: 50%;  
  height: 100px;  
  background-color: lightblue;  
}
```

HTML:

```
<div class="contenedor">  
  <div class="caja"></div>  
</div>
```

1. El contenedor mide:
600px
2. La caja tiene:
width: 50%
3. Entonces:
 $50\% \text{ de } 600\text{px} = 300\text{px}$

La caja se mostrará con **300px de ancho**.

Unidades de medida CSS

Hojas de estilo en cascada

Relativas: porcentajes. Ejemplo 3 – Dos columnas con porcentaje

```
.contenedor {  
  width: 600px;  
  border: 2px solid black;  
}  
  
.columna {  
  width: 50%;  
  float: left;  
  padding: 10px;  
  box-sizing: border-box; /*para evitar  
que el padding aumente el ancho total de la  
columna.*/  
}  
  
.columna:nth-child(1) {  
  background-color: lightblue;  
}  
  
.columna:nth-child(2) {  
  background-color: lightgreen;  
}
```

```
<div class="contenedor">  
  
  <div class="columna">  
    <h2>Columna izquierda</h2>  
    <p>Contenido de la primera columna.</p>  
  </div>  
  
  <div class="columna">  
    <h2>Columna derecha</h2>  
    <p>Contenido de la segunda columna.</p>  
  </div>  
  
</div>
```

Columna izquierda	Columna derecha
300px	300px

CSS Grid Layout

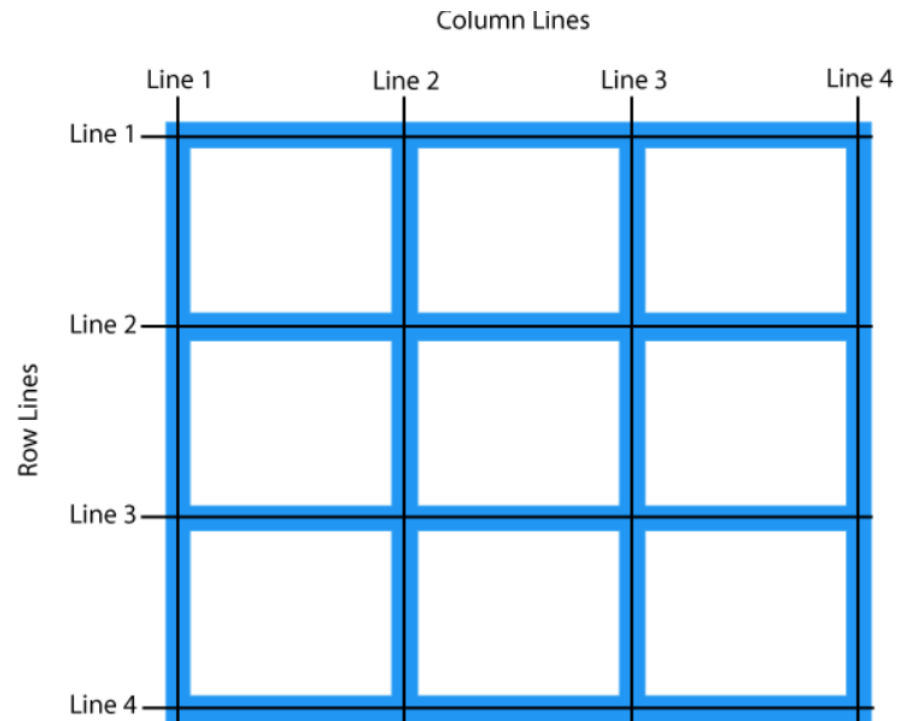
Hojas de estilo en cascada

- CSS Grid layout contiene funciones de diseño que permiten dividir una página en áreas o regiones principales y definir su relación en términos de tamaño, posición y capas entre partes de un control construido a partir de primitivas HTML. Es de dos dimensiones.

```
.grid-container {  
  display: grid;  
  grid-template-columns: auto auto auto;  
  background-color: #2196F3;  
  padding: 10px;  
}
```



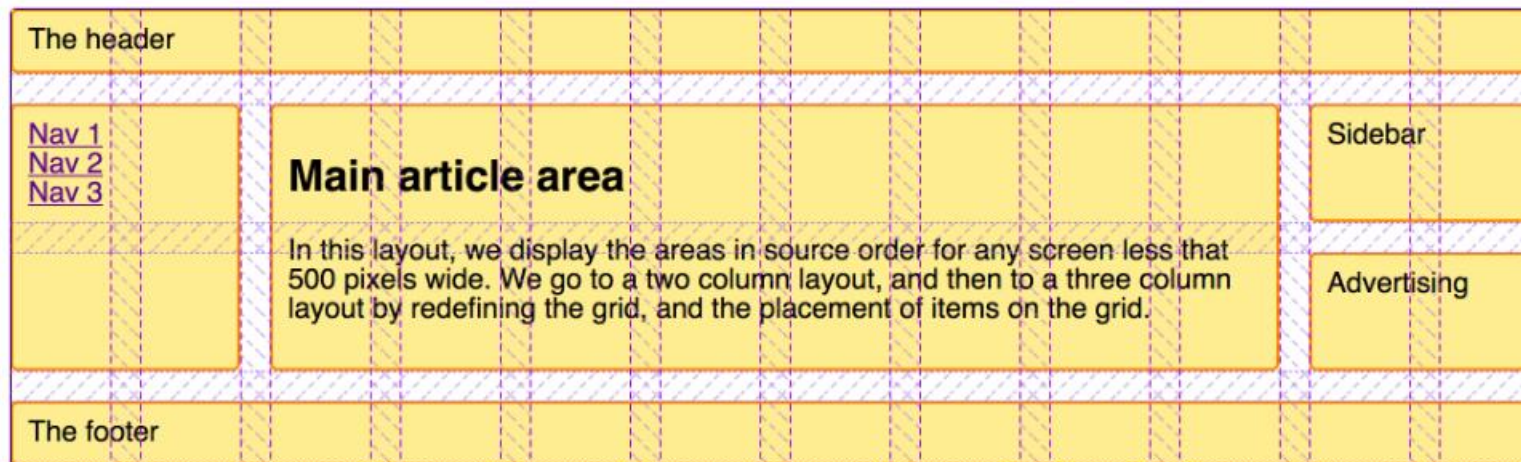
```
<div class="grid-container">  
  <div class="grid-item">1</div>  
  <div class="grid-item">2</div>  
  <div class="grid-item">3</div>  
  <div class="grid-item">4</div>  
  <div class="grid-item">5</div>  
  <div class="grid-item">6</div>  
  <div class="grid-item">7</div>  
  <div class="grid-item">8</div>  
  <div class="grid-item">9</div>  
</div>
```



CSS Grid Layout

Hojas de estilo en cascada

- CSS Grid layout fue introducido por Phil Cupp en 2011 en Microsoft para Internet Explorer 10. Google lo añade en Chromium 56 hasta 2017. Lo que hizo que Firefox y Safari hicieran lo mismo.
- Aún no es un estándar oficial W3C, es una recomendación candidata.



<https://www.w3.org/TR/css-grid-2/>

<https://caniuse.com/>

Regla @media

Hojas de estilo en cascada

- Le permiten a las paginas usar diferentes estilos CSS basándose en las características del dispositivo en el que el sitio es desplegado.
- Utiliza la @mediaregla para incluir un bloque de propiedades CSS solo si una determinada condición es verdadera.
- Podemos agregar un punto de interrupción donde ciertas partes del diseño se comportarán de manera diferente en cada lado del punto de interrupción.

Si la ventana del navegador tiene 600 px o menos, el color de fondo será azul claro:

```
@media only screen and (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Comentarios

Hojas de estilo en cascada

- Los comentarios son usados para añadir notas explicatorias o prevenir que el navegador interprete partes de la hoja de estilos.
- Los comentarios se pueden colocar en cualquier espacio en blanco que se permita en la hoja de estilos.

Sintaxis

```
/* Comentario */
```

Ejemplos

CSS

```
/* Comentario de una sola línea */
```

```
/*  
Un comentario  
que se extiende  
en varias  
líneas  
*/
```

Diseño Web Responsivo

Hojas de estilo en cascada

Introducción

Diseño Web Responsivo

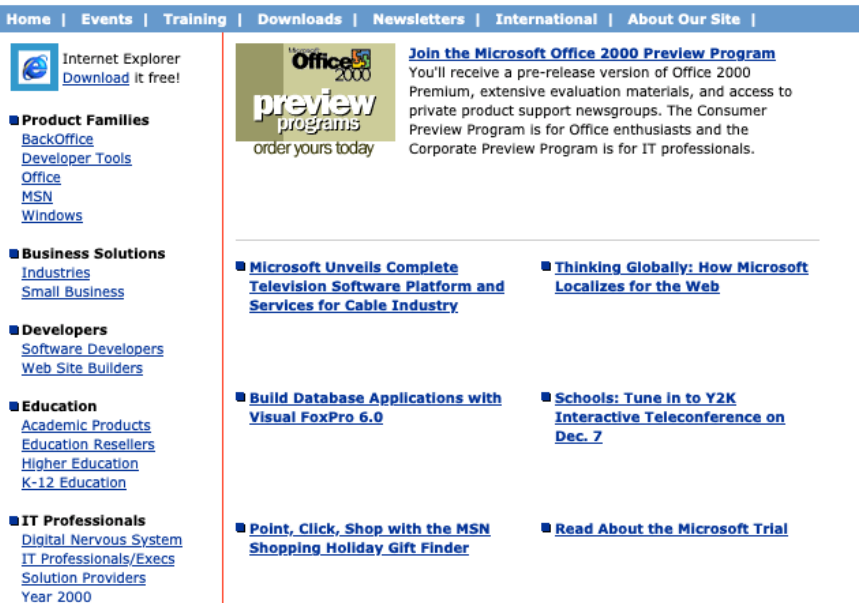
- En los primeros días de la web, la mayoría de la gente usaba computadoras de escritorio.
- Hoy en día, la Web está disponible en computadoras de escritorio, portátiles, tabletas, teléfonos plegables, refrigeradores y automóviles.
- La gente espera, con razón, que los sitios web se vean bien sin importar el dispositivo que utilicen. El diseño responsivo lo hace posible.
- El diseño responsivo no es el primer enfoque para diseñar sitios web. En los años previos al diseño responsivo, los diseñadores y desarrolladores web probaron muchas técnicas diferentes.

Diseño de ancho fijo

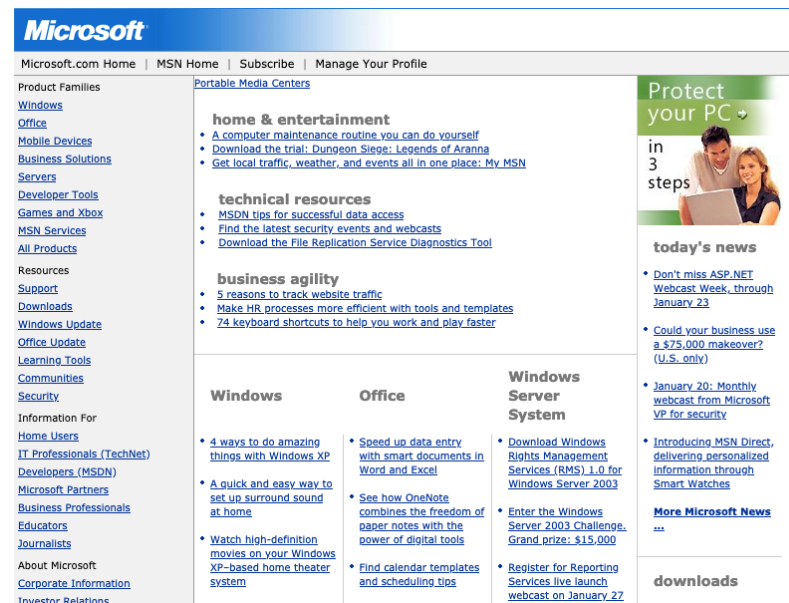
Diseño Web Responsivo

- A principios de la década de 1990, cuando la Web comenzaba a hacerse popular, la mayoría de los monitores tenían unas dimensiones de pantalla de **640 píxeles de ancho** por 480 píxeles de alto.
- En poco tiempo, la mayoría de las pantallas tenían unas dimensiones de 800 por 600 píxeles. Los diseños web cambiaron en consecuencia. Los diseñadores y desarrolladores empezaron a asumir que **800 píxeles era un valor predeterminado seguro**.
- Luego las pantallas volvieron a hacerse más grandes. **1024 por 768 píxeles** se convirtió en el valor predeterminado.

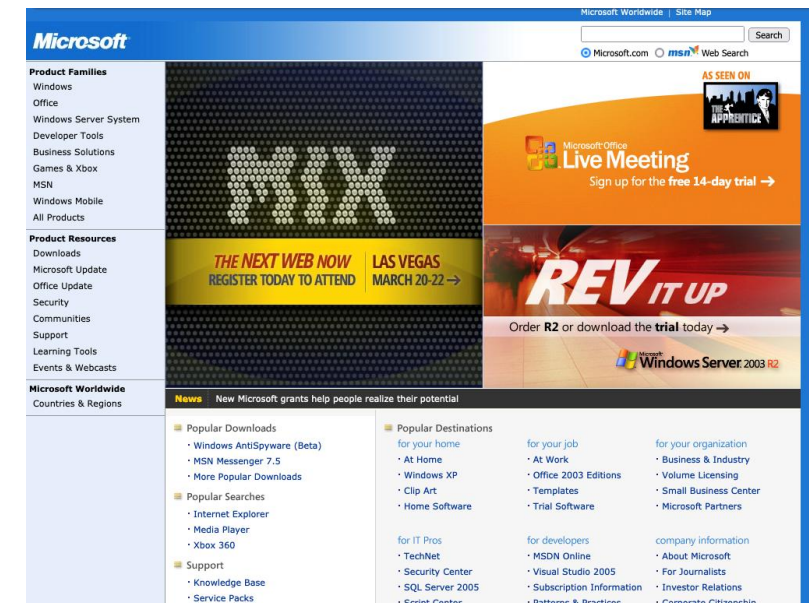
Finales de los 90 se diseñó para un ancho de 640 píxeles



Principios de los 2000 se diseñó para un ancho de 800 píxeles



Mediados de los 2000 se diseñó para un ancho de 1024 píxeles



Diseños líquidos (fluidos o flexibles)

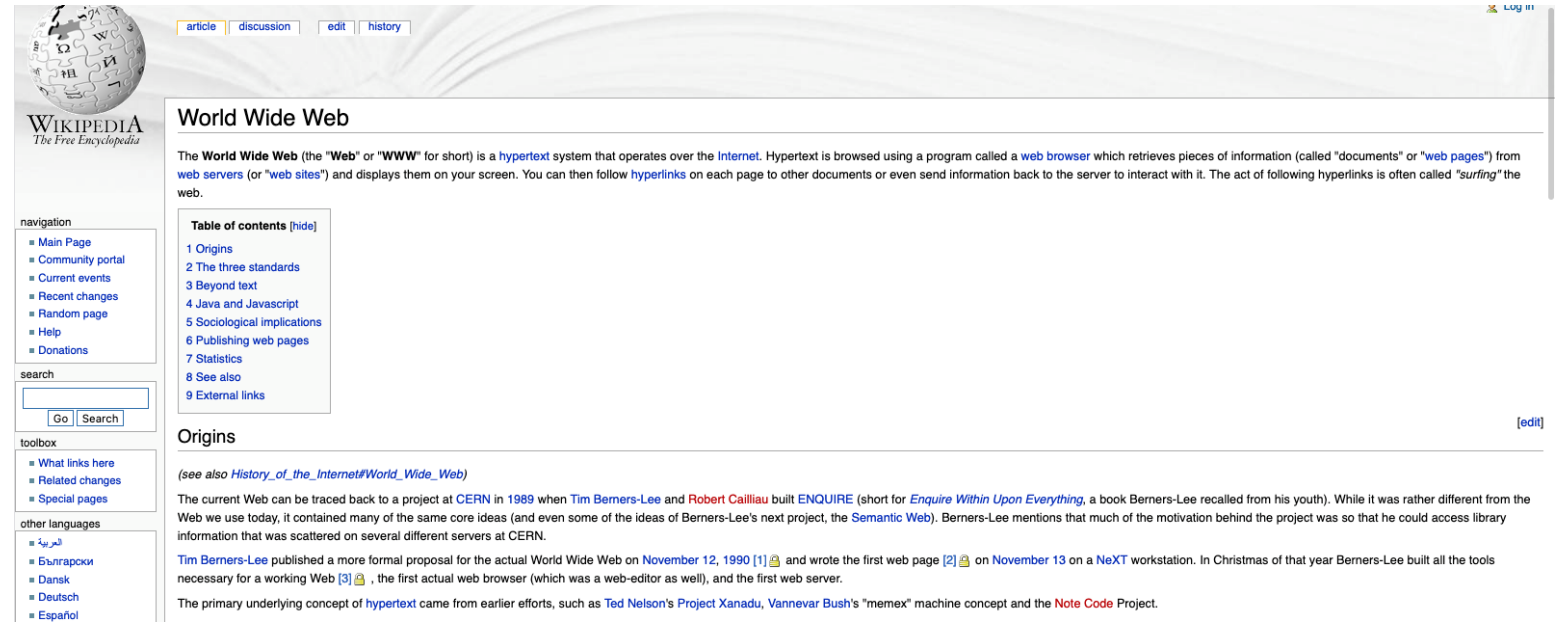
Diseño Web Responsivo

- En lugar de utilizar anchos fijos, se optó por crear un diseño flexible utilizando **porcentajes** para los anchos de sus columnas. Estos diseños funcionan en más situaciones que un diseño de ancho fijo que solo se ve bien en un tamaño específico.
- Estos se denominaron **diseños líquidos**. Pero si bien un diseño líquido se verá bien en una amplia gama de anchos, comenzará a empeorar en los extremos. En una pantalla ancha, el diseño parece estirado. En una pantalla estrecha, el diseño parece aplastado. Ambos escenarios no son ideales.

Diseño líquido de Wikipedia de mediados de la década de 2000 experimentado en una ventana estrecha del navegador



El diseño líquido de Wikipedia de mediados de la década de 2000 experimentado en una ventana amplia del navegador



Aparecen los dispositivos móviles

Diseño Web Responsivo

- **Pantallas pequeñas**

Con la llegada de los teléfonos móviles con navegadores web con todas las funciones, los diseñadores se enfrentaron a un dilema. ¿Cómo podían asegurarse de que sus diseños se vieran bien en una computadora de escritorio y en un teléfono móvil? Necesitaban una forma de diseñar su contenido para pantallas tan pequeñas como 240 píxeles de ancho y tan grandes como miles de píxeles de ancho.

- **Sitios separados**

Una opción es crear un subdominio separado para los visitantes móviles. Pero luego hay que mantener dos bases de código y diseños separados. Y para redirigir a los visitantes en dispositivos móviles, necesitaría realizar un rastreo de agente de usuario, que puede ser poco confiable y fácil de falsificar. Además, no existe una línea clara entre dispositivos móviles y no móviles. ¿A qué sitio envían tabletas?

Diseños adaptativos

Diseño Web Responsivo

- En lugar de tener sitios separados en diferentes subdominios, podría tener un solo sitio con dos o tres diseños de ancho fijo.
- Cuando las **media queries** llegaron por primera vez a CSS, abrieron la puerta a hacer que los diseños fueran más flexibles. Pero muchos desarrolladores todavía se sentían más cómodos haciendo diseños de ancho fijo. Una técnica implicaba cambiar entre un grupo de diseños de ancho fijo en anchos específicos. A esto se le llama **diseño adaptativo**.
- El diseño adaptativo permitió a los diseñadores proporcionar diseños que se veían bien en algunos tamaños diferentes, pero el diseño nunca se veía del todo bien cuando se veía entre esos tamaños. El problema del exceso de espacio persistía aunque no era tan grave como en un diseño de ancho fijo.

```
@media all and (min-width: 800px) {  
  article {  
    width: 540px;  
    float: left;  
  }  
  
  aside {  
    width: 250px;  
    float: left;  
    margin-left: 10px;  
  }  
}  
  
@media all and (min-width: 1200px) {  
  article {  
    width: 800px;  
    float: left;  
  }  
  
  aside {  
    width: 350px;  
    float: left;  
    margin-left: 50px;  
  }  
}  
  
@media all and (min-width: 1600px) {  
  h1 {  
    width: 400px;  
    float: left;  
  }  
}
```

Diseño web responsivo

Diseño Web Responsivo

- Si los diseños adaptables son una combinación de consultas de medios y diseños de ancho fijo, el diseño web responsivo es una combinación de consultas de medios y diseños líquidos.
- El término fue acuñado por Ethan Marcotte en un artículo de [A List Apart](#) en 2010.
- El objetivo es que el diseño y las imágenes de un sitio responsivo se verían bien en cualquier dispositivo con el mismo código.

```
@media all and (min-width: 800px) {  
  article {  
    width: 66%;  
    float: left;  
  }  
  
  aside {  
    width: 33%;  
    float: right;  
  }  
}  
  
@media all and (min-width: 1600px) {  
  h1 {  
    width: 22.5%;  
    float: left;  
  }  
  
  article {  
    width: 50%;  
    float: left;  
  }  
  
  aside {  
    width: 22.5%;  
    float: right;  
  }  
}
```

Definición

Diseño Web Responsivo

- El diseño Web responsivo (RWD) es una aproximación cuyo objetivo es permitir que paginas Web de escritorio sean visualizadas de acuerdo al tamaño de la pantalla o al navegador con el que se accede a ellas.
- Las tareas del diseño Web responsivo incluyen ofrecer el mismo soporte para una variedad de dispositivos dado un mismo sitio Web.
- El contenido, diseño y rendimiento son necesarios en todos los dispositivos para asegurar la usabilidad y satisfacción del usuario.

<https://web.dev/learn/design/responsive-images/>

Ventana de visualización (viewport)

Diseño Web Responsivo

- Los navegadores de teléfonos móviles tienen que lidiar con sitios web diseñados con diseños de ancho fijo para pantallas más amplias.
- Por defecto, los navegadores móviles asumen que **980 píxeles** es el ancho para el que la gente diseña. Entonces, incluso si usa un diseño líquido, el navegador móvil aplicará un ancho de 980 píxeles y luego **reducirá** la página web renderizada al ancho real de la pantalla.
- Si está utilizando un diseño responsivo, debe indicarle al navegador que no realice esa escala. Puedes hacerlo con un metaelemento en el head de la página web:

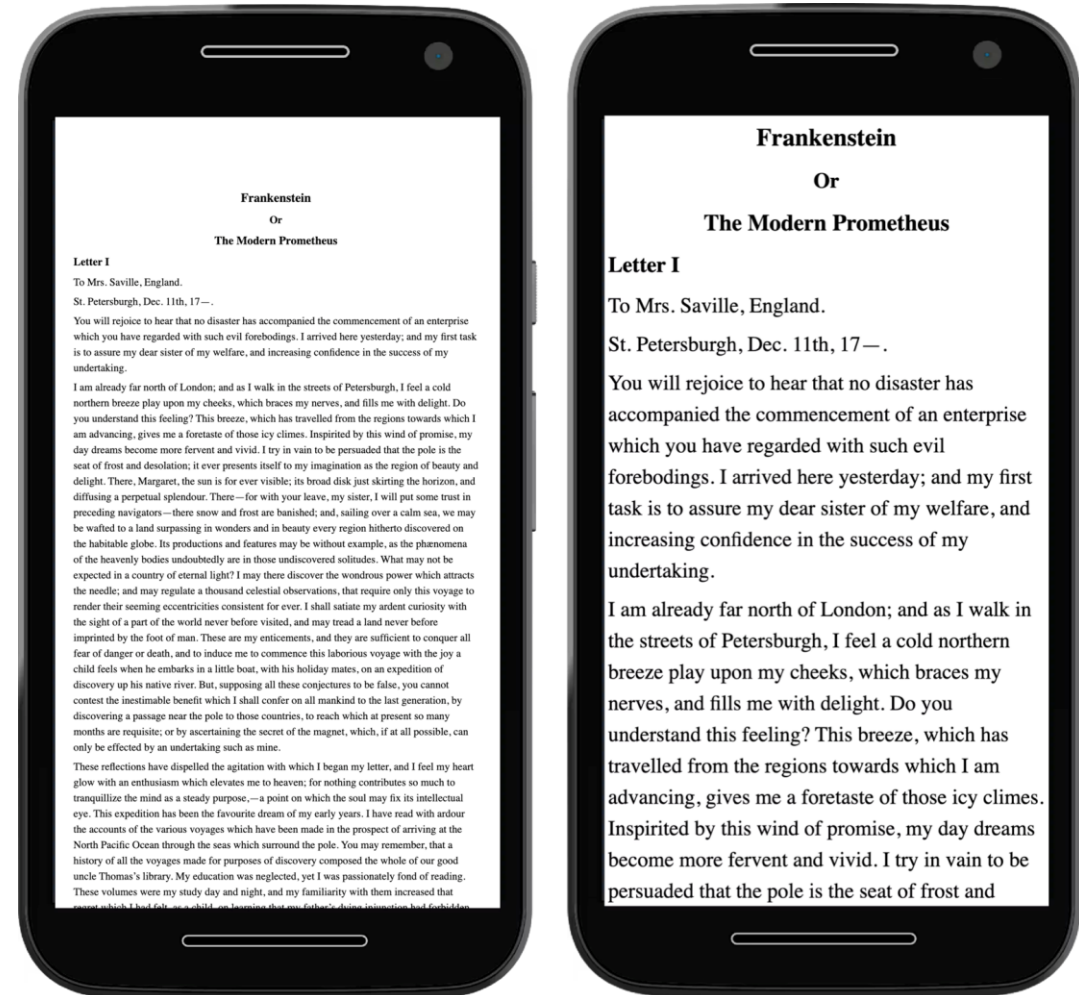
```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Ventana de visualización (viewport)

Diseño Web Responsivo

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

- Hay dos valores, separados por comas. El primero es `width=device-width`. Esto le indica al navegador que asuma que el ancho del sitio web es el mismo que el ancho del dispositivo (en lugar de asumir que el ancho del sitio web es de 980 píxeles).
- El segundo valor es `initial-scale=1`. Esto le dice al navegador cuánto o qué poco escalar debe hacer. Con un diseño responsivo, no querrás que el navegador escale en absoluto.



Diseño web responsivo

Diseño Web Responsivo

- Ethan Marcotte definió tres criterios para el diseño responsivo:
 - **Fluid media: Imágenes fluidas.**
 - **Fluid grids: Grid fluidos**
 - **Media queries: Regla @media**

Imágenes flexibles

Diseño Web Responsivo

- Son escaladas en unidades relativas, para prevenir que se desplieguen fuera de su elemento contenedor
- En CSS3 para obtener una imagen responsiva que adapta su resolución solo de ser necesario, de acuerdo al ancho de la pantalla:

```
.responsive {  
  width: 100%;  
  height: auto;  
}
```

```
.responsive {  
  max-width: 100%;  
  height: auto;  
}
```

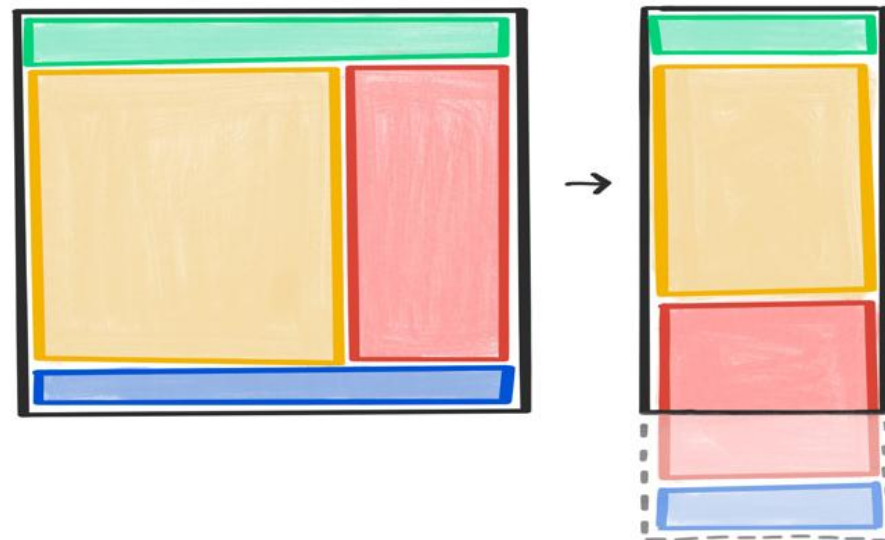
```
.responsive {  
  width: 100%;  
  max-width: 400px;  
  height: auto;  
}
```

https://www.w3schools.com/howto/howto_css_image_responsive.asp

Grids Fluidos

Diseño Web Responsivo

- Es un método de diseño de páginas web en dos dimensiones. Permite distribuir el contenido en filas y columnas, y tiene muchas características que facilitan la creación de diseños complejos.
- **¿Qué es la cuadrícula o Grid?**
 - Una cuadrícula es un conjunto de líneas horizontales y verticales que crean un patrón sobre el que podemos alinear nuestros elementos de diseño. Las cuadrículas nos ayudan a crear diseños de página en que los elementos no saltan ni cambian de ancho cuando nos movemos de una página a otra, y así proporcionan a nuestras páginas web un aspecto más coherente.



Grids Fluidos

Diseño Web Responsivo

- Adapta el tamaño de los elementos de forma relativa a partir de porcentajes, en vez de utilizar unidades absolutas como los pixeles.
- Una pagina Web responsiva normalmente cuenta con 12 columnas para ordenar contenido, la idea es crear una clase para ocupar de 1 a 12 columnas, adaptando el porcentaje del ancho.
- Al igual que las tablas, el grid layout permite a un autor alinear elementos en columnas y filas.
- Por ejemplo, los elementos secundarios de un contenedor de cuadrícula podrían posicionarse para que se solapen y se superpongan, de forma similar a los elementos posicionados en CSS.

Grids Fluidos

Diseño Web Responsivo

Responsive Grid

W3.CSS supports a 12 column responsive fluid grid.

Resize the page to see the effect!

1	2	3	4	5	6	7	8	9	10	11	12
This part will occupy 12 columns on a small screen, 4 on a medium screen, and 3 on a large screen.			This part will occupy 12 columns on a small screen, 8 on a medium screen, and 9 on a large screen.								
1	2	3	4	5	6	7	8	9	10	11	12

```

/* For desktop: */
.col-1 {width: 8.33%;}
.col-2 {width: 16.66%;}
.col-3 {width: 25%;}
.col-4 {width: 33.33%;}
.col-5 {width: 41.66%;}
.col-6 {width: 50%;}
.col-7 {width: 58.33%;}
.col-8 {width: 66.66%;}
.col-9 {width: 75%;}
.col-10 {width: 83.33%;}
.col-11 {width: 91.66%;}
.col-12 {width: 100%;}

```

```

@media only screen and (max-width: 768px) {
  /* For mobile phones: */
  [class*="col-"] {
    width: 100%;
  }
}

```

https://www.w3schools.com/w3css/w3css_grid.asp

<https://www.w3schools.com/w3css/4/w3.css>

Regla @media

Diseño Web Responsivo

- Le permiten a las paginas usar diferentes estilos CSS basándose en las características del dispositivo en el que el sitio es desplegado.
- Utiliza la @mediaregla para incluir un bloque de propiedades CSS solo si una determinada condición es verdadera.
- Podemos agregar un punto de interrupción donde ciertas partes del diseño se comportarán de manera diferente en cada lado del punto de interrupción.

Si la ventana del navegador tiene 600 px o menos, el color de fondo será azul claro:

```
@media only screen and (max-width: 600px) {  
  body {  
    background-color: lightblue;  
  }  
}
```

Regla @media

Diseño Web Responsivo

- Puede agregar tantos puntos de interrupción como desee.

```
/* Extra small devices (phones, 600px and down) */  
@media only screen and (max-width: 600px) {...}  
  
/* Small devices (portrait tablets and large phones, 600px and up) */  
@media only screen and (min-width: 600px) {...}  
  
/* Medium devices (landscape tablets, 768px and up) */  
@media only screen and (min-width: 768px) {...}  
  
/* Large devices (laptops/desktops, 992px and up) */  
@media only screen and (min-width: 992px) {...}  
  
/* Extra large devices (large laptops and desktops, 1200px and up) */  
@media only screen and (min-width: 1200px) {...}
```



Escritorio



Tableta



Teléfono

https://www.w3schools.com/cssref/css3_pr_mediaquery.php

Regla @media

Diseño Web Responsivo

- Orientación: Retrato/Paisaje

```
@media only screen and (orientation: landscape) {  
  body {  
    background-color: lightblue;  
  }  
}
```

- Ocultar elementos

```
/* If the screen size is 600px wide or less, hide the element */  
@media only screen and (max-width: 600px) {  
  div.example {  
    display: none;  
  }  
}
```

- Cambiar el tamaño de fuente

```
/* If the screen size is 601px or more, set the font-size of <div> to 80px */  
@media only screen and (min-width: 601px) {  
  div.example {  
    font-size: 80px;  
  }  
}  
  
/* If the screen size is 600px or less, set the font-size of <div> to 30px */  
@media only screen and (max-width: 600px) {  
  div.example {  
    font-size: 30px;  
  }  
}
```

CSS Grid Layout

Diseño Web Responsivo

- CSS Grid layout contiene funciones de diseño que permiten dividir una página en áreas o regiones principales y definir su relación en términos de tamaño, posición y capas entre partes de un control construido a partir de primitivas HTML. A diferencia de flexbox, grid es de dos dimensiones.

```
.wrapper {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  grid-auto-rows: 100px;  
}
```

```
.box1 {  
  grid-column: 1;  
  grid-row: 2;  
}
```

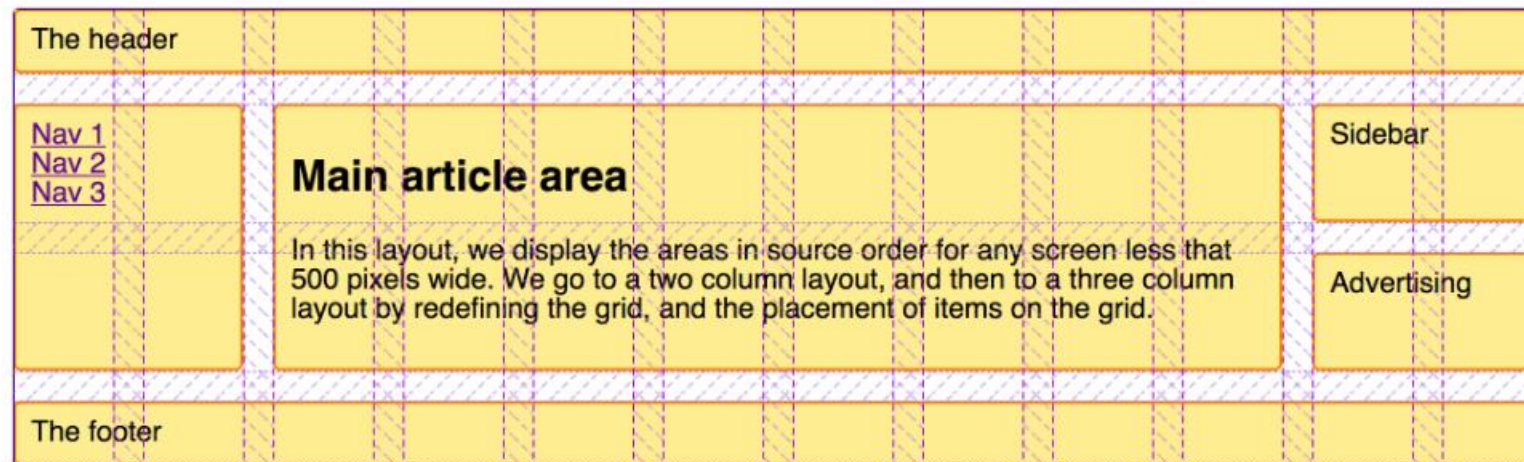
```
<div class="wrapper">  
  <div class="box box1"><a href="">One</a></div>  
  <div class="box box2"><a href="">Two</a></div>  
  <div class="box box3"><a href="">Three</a></div>  
  <div class="box box4"><a href="">Four</a></div>  
  <div class="box box5"><a href="">Five</a></div>  
</div>
```



CSS Grid Layout

Diseño Web Responsivo

- CSS Grid layout fue introducido por Phil Cupp en 2011 en Microsoft para Internet Explorer 10. Google lo añade en Chromium 56 hasta 2017. Lo que hizo que Firefox y Safari hicieran lo mismo.
- Aún no es un estándar official W3C, es una recomendación candidata.



<https://www.w3.org/TR/css-grid-2/>

<https://caniuse.com/>

Animaciones CSS

Diseño Web Responsivo

- Permite la animación de elementos HTML sin usar JavaScript o Flash.
- Una animación permite que un elemento cambie gradualmente de un estilo a otro.
- Puedes cambiar tantas propiedades CSS como quieras, tantas veces como quieras.
- Para utilizar la animación CSS, primero debe especificar algunos fotogramas clave para la animación. **Regla @keyframes.**
- Los fotogramas clave contienen qué estilos tendrá el elemento en determinados momentos.

- @keyframes
- animation-name
- animation-duration
- animation-delay
- animation-iteration-count
- animation-direction
- animation-timing-function
- animation-fill-mode
- animation

```
/* The animation code */
@keyframes example {
  from {background-color: red;}
  to {background-color: yellow;}
}

/* The element to apply the animation to */
div {
  width: 100px;
  height: 100px;
  background-color: red;
  animation-name: example;
  animation-duration: 4s;
}
```



Consejos generales

Diseño Web Responsivo

- No utilizar elementos muy anchos con un ancho predefinido, esto causa que la ventana de visualización haga scroll horizontal.
- El contenido no deberá estar sujeto a un ancho de ventana de visualización particular. Las dimensiones entre pantallas varían mucho entre dispositivos.
- Siempre que se pueda, se recomienda diseñar para móviles primero. Dado que las pantallas pequeñas pueden presentar mayores problemas que las grandes, de esta forma se considera primero el caso especial.
- Utilizar un framework de diseño web.

Mobile First

Diseño Web Responsivo

- Si la audiencia principal de la aplicación web son los dispositivos con pantallas pequeñas. Se debe diseñar usando código Mobile First.
- Mobile First significa diseñar para dispositivos móviles antes de diseñar para computadoras de escritorio o cualquier otro dispositivo (esto hará que la página se muestre más rápido en dispositivos más pequeños).
- Esto significa que debemos realizar algunos cambios en nuestro CSS.
- En lugar de cambiar los estilos cuando el ancho sea menor que 768px, deberíamos cambiar el diseño cuando el ancho sea mayor que 768px.

Frameworks de CSS

Diseño Web Responsivo



<https://tailwindcss.com/>

<https://materializecss.com/>

<https://get.foundation/sites/docs/>

<https://getbootstrap.com/docs/5.3/getting-started/introduction/>

<https://semantic-ui.com/introduction/getting-started.html>

<https://mui.com/material-ui/getting-started/>



Resumen

Tecnologías web

- HTML es un lenguaje muy sencillo compuesto de elementos, que se pueden aplicar a piezas de texto para darles un significado diferente en un documento (párrafos, lista con viñetas, tablas), estructura un documento en secciones lógicas (cabecera, columnas de contenido, menú de navegación), e incrusta contenido como imágenes y vídeos en una página.
- Las Hojas de estilo en cascada (del ingles Cascading Stylesheets CSS) es la siguiente tecnología que aprenderemos después de HTML.
- Mientras que HTML se utiliza para definir la estructura y la semántica del contenido, CSS se usa para darle estilo y posicionarlo visualmente.
- CSS se puede usar, por ejemplo, para cambiar la fuente, el color, el tamaño y el espaciado del contenido, para formar multiples columnas, añadir animaciones y otros elementos decorativos.

Preguntas de repaso

Tecnologías web

1. ¿Cuál de los siguientes es un elemento de bloque HTML?

a) `<ul>`

b) `<strong>`

c) `<span>`

d) `<div>`

2. ¿Cuáles son los tres fundamentos de desarrollo web?

a) HTML, CSS, Javascript

b) Rejillas fluidas,
medios fluidos, media
queries

c) HTML, Editor,
Servidor web

d) Frontend, Backend,
middleware

3. ¿Cuál es la sintaxis de comentarios para CSS?

a) `<!-- -->`

b) `/* */`

c) `//`

d) `#- -#`

Preguntas de repaso

Tecnologías web

4. Lenguaje empleado para modificar la apariencia o presentación del contenido de una página Web.

a) HTML

b) IMG

c) RWD

d) CSS

5. Es un lenguaje de marcado que le dice a los navegadores web cómo estructurar las páginas web que estás visitando.

a) HTML

b) IMG

c) RWD

d) CSS

6. Existen dos tipos de unidades en reglas CSS.

a) Atributos y clases

b) Relativas y absolutas

c) Marcado y cascada

d) Media queries y grids

Preguntas de repaso

Tecnologías web

7. El selector universal * de CSS aplica cuándo es necesario aplicar un estilo con todos los elementos de una página Web.

a) Verdadero

b) Falso

8. Los media queries son útiles cuando es necesario aplicar estilos dependiendo de un tipo de dispositivo en particular como impresoras, proyectores o pantallas.

a) Verdadero

b) Falso

9. ¿Cuáles son tres distintas formas de aplicar estilos CSS a elementos HTML?

a) Eterna, por SSL y en un XML

b) Relativas, flexibles y absolutas

c) En línea, Incrustado en la cabecera y hoja de estilos externa

d) Usando media queries, grids y JavaScript



Preguntas